

Laboratório de Concorrência · Aula 10

João Paulo Traguetta Rufino · RA 24000674-2 · Turma ESOF6S A

Gerado em 10/10/2026, 17:27:51 · tempo registrado no cronômetro: 36:03

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação: um processo depende do resultado do outro. Ex.: o montador de pratos espera o molho ficar pronto.

Competição: os processos disputam um recurso exclusivo. Ex.: duas pessoas, um único banheiro.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Valor esperado: 2.000.000

Valores obtidos nas três execuções: variam conforme o resultado de cada execução do programa. Você precisa executar o código para preencher os valores reais.

1ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1031790

2ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1241889

3ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1021424

Saída do programa

não respondido



```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console

esperado: 2000000
obtido: 1021424

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não necessariamente. Os valores obtidos podem ser menores que 2.000.000 e variar entre as execuções devido à condição de corrida. Isso acontece porque as duas threads acessam e modificam a mesma variável ao mesmo tempo, podendo sobrescrever as atualizações uma da outra. O resultado depende de como as operações são intercaladas pelo processador e pelo sistema operacional.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

A atualização perdida acontece quando as duas threads leem o mesmo valor do contador antes de qualquer uma gravar o resultado. Por exemplo:

1. Ler: a thread 1 e a thread 2 leem o valor 10.
2. Somar: ambas somam 1 ao valor lido, obtendo 11.
3. Gravar: as duas gravam 11 no contador.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com $n = 1000$, o resultado pode ser 2.000 ou um valor menor. Como há menos operações, a condição de corrida pode ocorrer com menor frequência, mas isso não significa que o problema foi resolvido.

Esse erro é perigoso em sistemas reais porque pode passar despercebido durante os testes e aparecer em produção, quando há mais usuários e operações simultâneas. Em sistemas financeiros, por exemplo, isso pode causar inconsistências em saldos, estoques ou registros. Para evitar o problema, é possível utilizar mecanismos de sincronização, como `synchronized`, garantindo que apenas uma thread por vez atualize o contador.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

ABABAB → 6

Saída do programa

não respondido

```

1 # Prática 2 - Todas as respostas possíveis da Figura 13.1 (Sebesta, p. 548)
2 # Cole em https://onecompiler.com/python
3 # TOTAL começa em 3. A tarefa A faz TOTAL + 1 e a tarefa B faz TOTAL * 2.
4 # Cada tarefa tem 3 passos: 1) ler TOTAL 2) calcular 3) gravar TOTAL.
5 from itertools import combinations
6
7
8 def executa(ordem):
9     total = 3
10    reg = {"A": 0, "B": 0}
11    passo = {"A": 0, "B": 0}
12    for t in ordem:
13        if passo[t] == 0:
14            reg[t] = total
15        elif passo[t] == 1:
16            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2
17        else:
18            total = reg[t]
19            passo[t] += 1
20    return total
21
22
23 print("AAABBB ->", executa("AAABBB")) # A inteira, depois B inteira
24 print("BBBAAA ->", executa("BBBAAA")) # B inteira, depois A inteira
25 print("ABABAB ->", executa("ABABAB")) # alternando passo a passo
26
27 resultados = {}
28 for posicoes in combinations(range(6), 3):
29     ordem = "".join("A" if i in posicoes else "B" for i in range(6))
30     resultados.setdefault(executa(ordem), []).append(ordem)
31
32 print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
33 for valor in sorted(resultados):
34     ordens = resultados[valor]
35     print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")
  
```

AAABBB -> 8
 BBBAAA -> 7
 ABABAB -> 6
 Ordens possíveis: 20
 TOTAL = 4: 9 ordens, por exemplo AAABBB
 TOTAL = 6: 9 ordens, por exemplo ABABAB
 TOTAL = 7: 1 ordens, por exemplo BBBAAA
 TOTAL = 8: 1 ordens, por exemplo AAABBB

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens de execução, pois existem 6 passos no total, sendo 3 de A e 3 de B. Aparecem 4 resultados diferentes: 4, 6, 7 e 8. O resultado 8 aparece em 6 ordens.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

A ordem BABABA produz o resultado 4:

1. B lê TOTAL = 3.
2. A lê TOTAL = 3.
3. B calcula $3 \times 2 = 6$.
4. A calcula $3 + 1 = 4$.
5. B grava TOTAL = 6.
6. A grava TOTAL = 4.

O resultado final é 4 porque A grava seu resultado depois de B, sobrescrevendo o valor 6. Assim, a atualização de B é perdida, mesmo que sua operação tenha sido calculada corretamente.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a versão sem sincronização será a mais rápida, pois não precisa controlar o acesso simultâneo à variável. Porém, ela pode apresentar um resultado incorreto devido à condição de corrida. As versões com `synchronized` e `AtomicInteger` devem apresentar o resultado correto, mas podem ser mais lentas devido ao custo da sincronização.

Saída do programa

não respondido



```
1 // Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
2 // Cole em https://onecompiler.com/java
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Main {
6     static int semCuidado = 0, comSync = 0;
7     static AtomicInteger atomico = new AtomicInteger();
8
9     static synchronized void incSync() { comSync++; } // uma thread por vez
10
11     static long mede(Runnable corpo) throws InterruptedException {
12         Thread a = new Thread(corpo), b = new Thread(corpo);
13         long inicio = System.nanoTime();
14         a.start(); b.start(); a.join(); b.join();
15         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
16     }
17
18     public static void main(String[] args) throws InterruptedException {
19         int n = 1_000_000;
20         long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
21         long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
22         long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
23         System.out.println("esperado: " + 2 * n);
24         System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
25         System.out.println("synchronized: " + comSync + " em " + ms2 + " ms");
26         System.out.println("AtomicInteger: " + atomico.get() + " em " + ms3 + " ms");
27     }
28 }
```

Console Output:

```
esperado: 2000000
sem sincronizacao: 1092400 em 5 ms
synchronized: 2000000 em 147 ms
AtomicInteger: 2000000 em 25 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões `synchronized` e `AtomicInteger` devem apresentar o valor correto de 2.000.000. A versão sem sincronização pode apresentar um valor menor devido à condição de corrida.

A versão sem sincronização costuma ser mais rápida porque não precisa controlar o acesso simultâneo à variável. Já `synchronized` e `AtomicInteger` precisam garantir que as atualizações sejam realizadas corretamente, o que gera um custo adicional de processamento. Os tempos variam conforme o computador e a execução.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um `AtomicInteger` em cada saldo resolve? Por quê? O que seria preciso?

Não. Um AtomicInteger em cada saldo garante que cada atualização individual seja atômica, mas não garante que a transferência inteira aconteça como uma única operação. Outra thread poderia acessar os saldos entre a retirada e o depósito, causando inconsistências.

Seria necessário sincronizar a operação completa, garantindo que a retirada de uma conta e o depósito na outra aconteçam juntos, por exemplo, usando um bloqueio (synchronized) que proteja as duas operações. Em um sistema bancário real, também seria importante utilizar uma transação no banco de dados para garantir a atomicidade e a consistência da transferência.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não. As mensagens "A terminou" e "B terminou" provavelmente não vão aparecer, porque as duas threads podem ficar presas em um deadlock. A thread A bloqueia o recurso X e espera Y, enquanto a thread B bloqueia Y e espera X. Assim, nenhuma consegue terminar.

Saídas (antes e depois da correção)

não respondido

```
1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); } }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse" : "IMPASSE: " + presas.length + " threads presas");
23        System.exit(0); // encerra o programa
24    }
25 }
26 }
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A bloqueou o recurso X e ficou esperando o recurso Y, que estava bloqueado pela thread B. Ao mesmo tempo, a thread B bloqueou Y e ficou esperando X, que estava bloqueado pela thread A. Assim, nenhuma das duas conseguia continuar, causando um impasse (deadlock).

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem evita que uma thread fique esperando um recurso enquanto segura outro que a segunda thread precisa. Dessa forma, elimina-se o ciclo de espera que causa o impasse nesse exemplo. Um exemplo real seria um sistema bancário em que duas operações precisam acessar duas contas simultaneamente. Se cada operação bloquear as contas em uma ordem diferente, elas podem ficar esperando uma pela outra. Definir uma ordem única para bloquear as contas evita esse tipo de impasse.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

- 3 trabalhadores: aproximadamente 200 ms.
- 1 trabalhador: aproximadamente 600 ms.
- 6 trabalhadores: aproximadamente 100 ms.

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

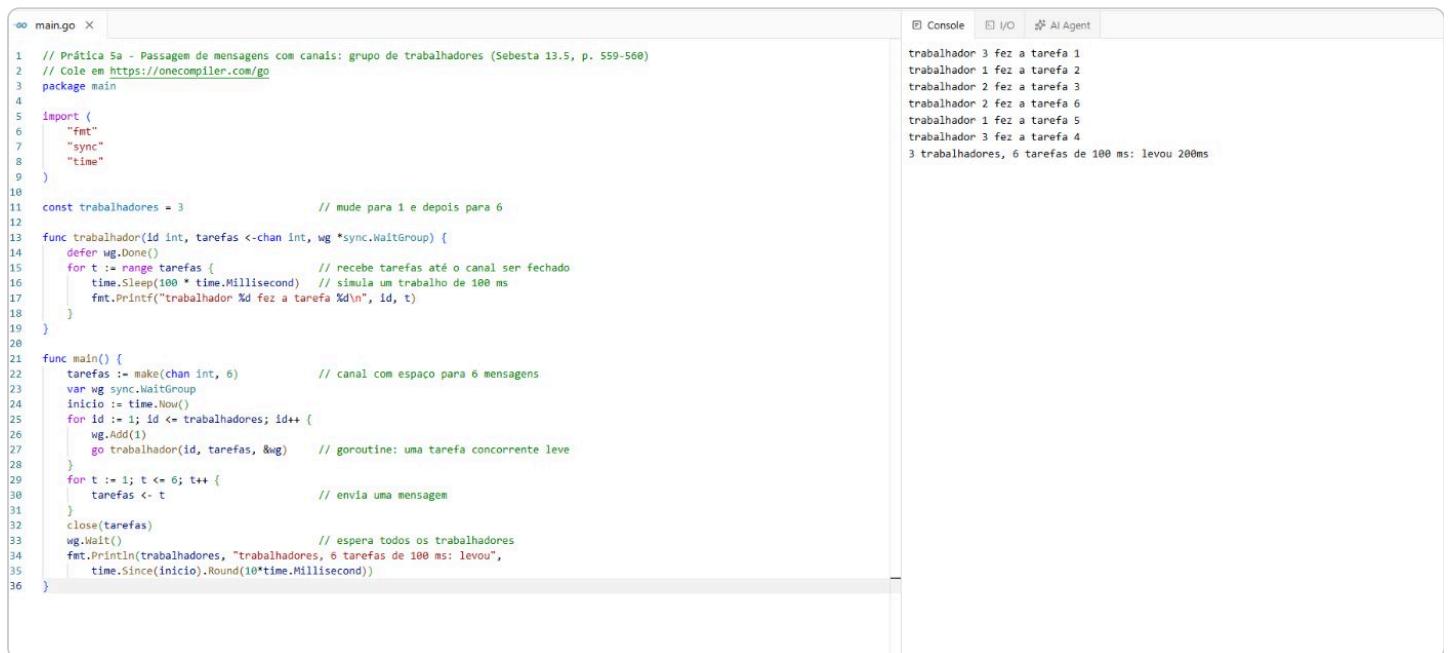
200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

não respondido



The screenshot shows a Go program in a text editor with a console output window. The program is a benchmark for message passing between workers. It defines a channel for tasks and a group of workers. Each worker receives tasks from the channel and prints a message. The main function creates a channel, adds workers, and sends tasks. The output shows the sequence of tasks being processed by the workers and the total time taken.

```
1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 3 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }
```

Console Output:

```
trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 2 fez a tarefa 6
trabalhador 1 fez a tarefa 5
trabalhador 3 fez a tarefa 4
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, as seis tarefas são executadas sequencialmente, levando cerca de 600 ms. Com 3 trabalhadores, cada um executa aproximadamente duas tarefas, levando cerca de 200 ms. Com 6 trabalhadores, cada um executa uma tarefa, levando cerca de 100 ms.

O programa não precisa de synchronized nem de trava porque as tarefas são distribuídas por canais, que permitem a comunicação entre goroutines sem precisar compartilhar diretamente uma variável para controlar o trabalho.

Saídas (antes e depois da troca)

não respondido

```

1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int)           // canal SEM espaço (sem buffer)
9     ch <- 1                       // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }

```

fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
/box/main.go:9 +0x38
exit status 2

Debug this error

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não possui espaço para armazenar mensagens. Quando o programa envia o valor 1, fica esperando que outra goroutine receba a mensagem, mas não existe nenhuma outra goroutine para fazer isso.

Ao utilizar `make(chan int, 1)`, o canal passa a ter espaço para armazenar uma mensagem. Assim, o valor 1 é armazenado e depois recebido pelo próprio programa, permitindo que ele imprima o valor e termine normalmente.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acredito que o programa apresentará um erro de compilação, porque o Rust não permite que a thread altere diretamente uma variável de main sem garantir a segurança do compartilhamento. O compilador impede esse acesso para evitar problemas de concorrência e memória.

Saídas (o erro e o resultado com move)

não respondido

```

1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(|| {
8         contador += 1;           // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }

```

note: function requires argument type to outlive `static`
--> HelloWorld.rs:7:13

```

7 |     let t = thread::spawn(|| {
8 |         contador += 1;           // a thread altera a variável de main
9 |     });

```

help: to force the closure to take ownership of `contador` (and any other referenced variables), use the `move` keyword

```

7 |     let t = thread::spawn(move || {
8 |         contador += 1;           // a thread altera a variável de main
9 |     });

```

error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
--> HelloWorld.rs:11:20

```

11 |     println!("{}", contador);

```

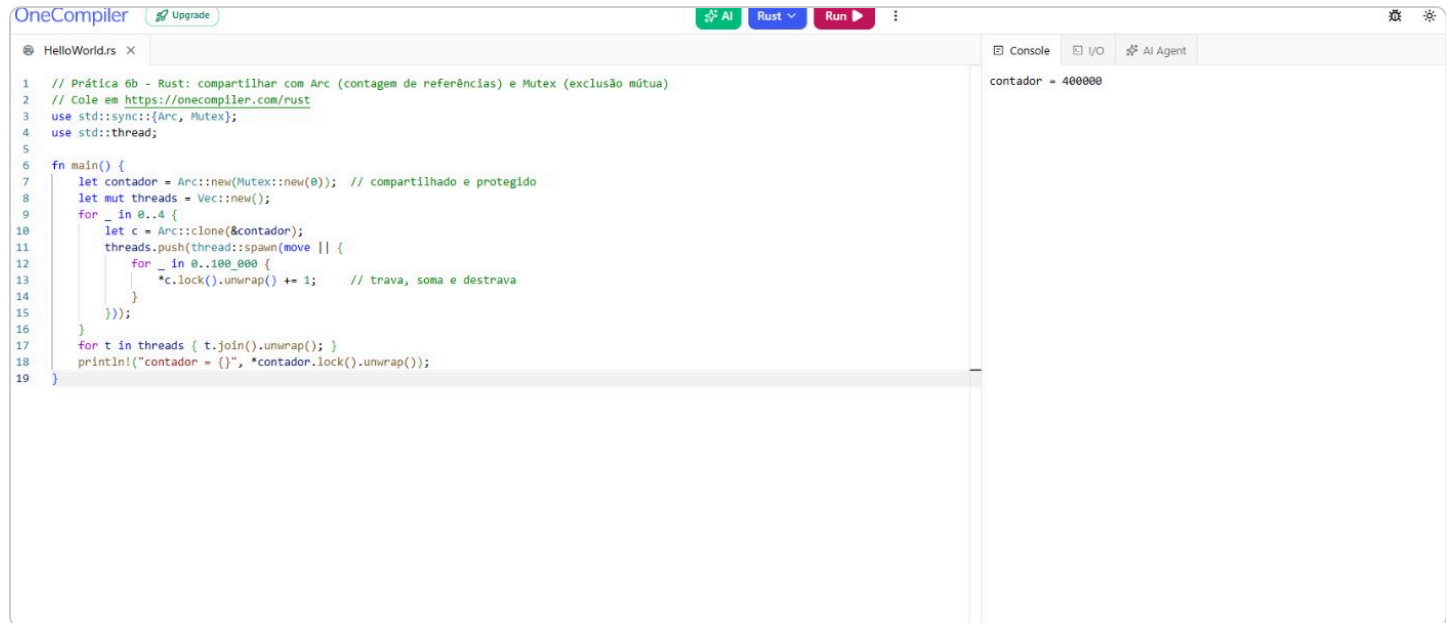
note: requirement that the value outlives `static` introduced here
--> /rustc/48a220ceaf04985c50990b14116b6d856af0985/library/std/src/thread/functions.rs:128:11

error: aborting due to 2 previous errors

Some errors have detailed explanations: E0373, E0502.
For more information about an error, try `rustc --explain E0373`.

Saída do programa

não respondido



The screenshot shows the OneCompiler web IDE. The editor displays a Rust file named 'HelloWorld.rs' with the following code:

```
1 // Prática 6b - Rust: compartilhar com Arc (contagem de referências) e Mutex (exclusão mútua)
2 // Cole em https://onecompiler.com/rust
3 use std::sync::{Arc, Mutex};
4 use std::thread;
5
6 fn main() {
7     let contador = Arc::new(Mutex::new(0)); // compartilhado e protegido
8     let mut threads = Vec::new();
9     for _ in 0..4 {
10         let c = Arc::clone(&contador);
11         threads.push(thread::spawn(move || {
12             for _ in 0..100_000 {
13                 *c.lock().unwrap() += 1; // trava, soma e destrava
14             }
15         }));
16     }
17     for t in threads { t.join().unwrap(); }
18     println!("contador = {}", *contador.lock().unwrap());
19 }
```

The console on the right shows the output: `contador = 400000`.

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O move transfere a propriedade do valor contador para a thread. Como o número inteiro (i32) é copiado, a thread altera sua própria cópia, enquanto a variável original continua valendo 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, a condição de corrida é descoberta durante a execução do programa e pode passar despercebida nos testes. Em Rust, o compilador detecta problemas de propriedade e compartilhamento inseguro antes da execução. A vantagem do Rust é evitar diversas falhas de concorrência antes que o programa seja executado. O custo é que o programador precisa aprender e respeitar as regras de propriedade, empréstimo e sincronização, o que pode tornar o desenvolvimento inicialmente mais complexo.

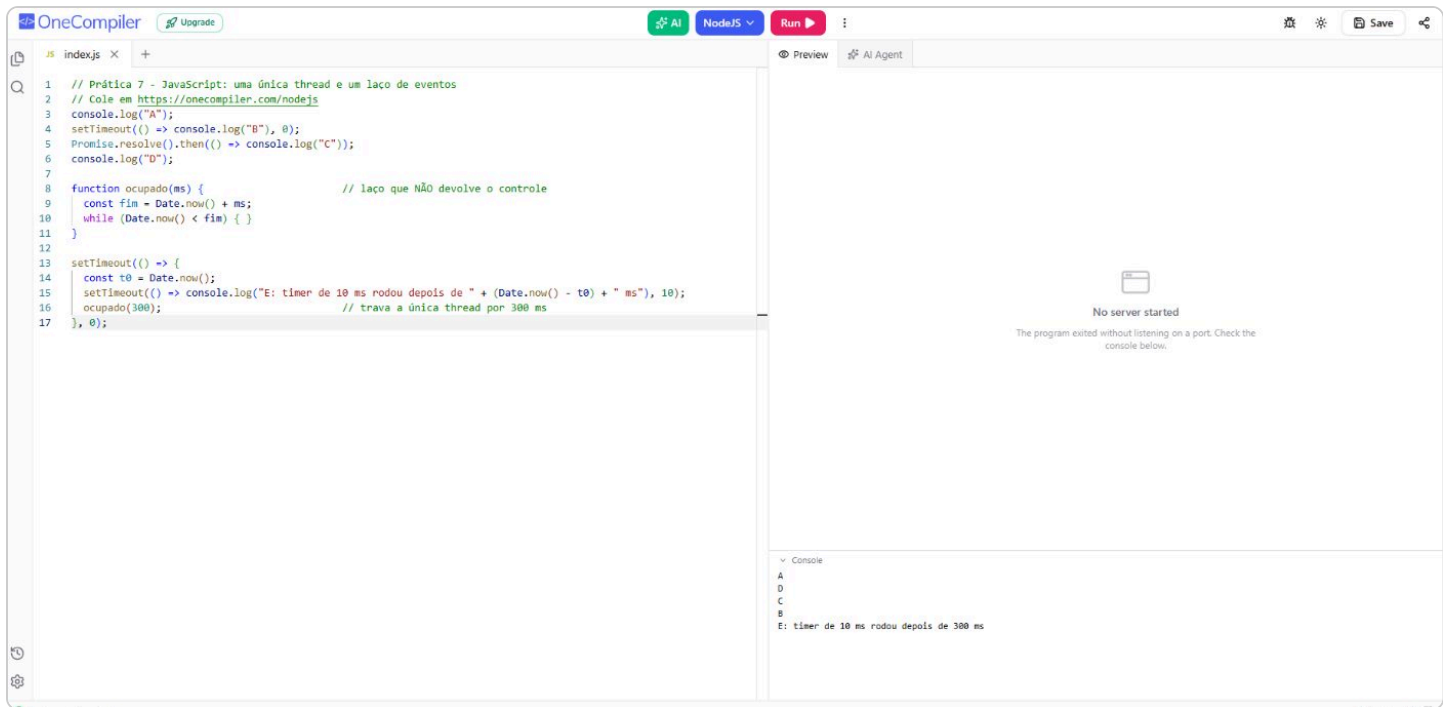
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A ordem esperada é A, D, C, B. Primeiro, A e D são impressas imediatamente. Depois, C é executada pela fila de microtarefas das Promises e, por último, B é executada pelo setTimeout, mesmo com tempo de 0 ms.

Saída do programa

não respondido



7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem é A, D, C, B. A e D são impressas imediatamente, durante a execução do código principal. C é executada pela fila de microtarefas das promessas, antes dos timers. B, apesar de ter setTimeout com tempo 0, precisa esperar o código atual e as microtarefas terminarem.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer demora mais porque a função ocupado(300) bloqueia a única thread por aproximadamente 300 ms. Durante esse período, o Node.js não consegue processar outros callbacks.

Em um servidor Node.js, cálculos pesados executados diretamente na thread principal podem atrasar as respostas de vários usuários ao mesmo tempo. Para evitar isso, é possível utilizar Worker Threads, processos separados ou transferir tarefas pesadas para serviços específicos.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O programador, com sincronização

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida, permitindo que as duas pessoas comprem o mesmo ingresso. Para evitar isso, usaria uma transação no banco de dados com bloqueio ou uma atualização atômica que garanta que apenas uma compra seja confirmada.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

A metodologia inovadora utilizada pelo professor